

Capítulo 1 — Introducción

1.1. Descripción del proyecto

TaskManager API es una aplicación REST desarrollada en el marco de la asignatura *Desarrollo Web II* (M1) del grado en Informática de la UMAIA, curso 2025/2026.

El sistema permite a los usuarios gestionar **proyectos personales** y las **tareas** asociadas a esos proyectos. Cada usuario dispone de un espacio aislado: únicamente puede acceder, crear, modificar y eliminar sus propios proyectos y tareas. Las credenciales y la autorización están gestionadas mediante tokens JWT.

La aplicación está pensada para funcionar como capa de servicios (backend) consumible por cualquier cliente web o móvil. En esta entrega se entrega documentada y verificada mediante una colección de Postman, pero el diseño es independiente del cliente.

1.2. Objetivos

El proyecto cumple con los requisitos obligatorios del enunciado:

- Arquitectura **REST**, utilizando los cuatro verbos HTTP (GET, POST, PUT, DELETE).
- **Tres recursos** distintos: **users**, **projects** y **tasks**.
- **Dos relaciones 1:n** entre los recursos: **users** → **projects** y **projects** → **tasks**.
- Representaciones en **JSON**.
- **Autenticación y autorización** mediante JWT, con control de acceso por propietario.
- **Logging** del usuario autenticado en consola para cada petición protegida.
- Documentación **OpenAPI 3.0** servida vía Swagger UI.
- **Colección Postman** para consulta de los recursos.
- Despliegue **multi-contenedor** con Docker Compose (MySQL + Node.js).
- **Base de datos MySQL** poblada con al menos 30 registros por tabla.

Adicionalmente, se han incorporado los siguientes elementos valorados en el enunciado:

- Uso del framework **Express**.
- **Aislamiento por propietario** aplicado en **projects** y **tasks** (un usuario solo accede a sus recursos).
- En el capítulo 4 se realiza una **comparación entre la implementación actual y los distintos flows de OAuth 2.0**.

1.3. Stack tecnológico

Capa	Tecnología
Servidor aplicativoal	Node.js 20 (Alpine)
Framework web	Express 5
Base de datos	MySQL 8.0
Driver MySQL	mysql2 (con pool de conexiones)
Autenticación	JSON Web Tokens (jsonwebtoken)

Capa	Tecnología
Hash de contraseñas	bcryptjs
Documentación API	OpenAPI 3.0 + swagger-ui-express
Contenerización	Docker + Docker Compose
Variables de entorno	dotenv
CORS	cors

1.4. Estructura del repositorio

- **api/**
 - `openapi.yaml` — Especificación OpenAPI 3.0
- **doc/**
 - `c1.md` — Introducción (este capítulo)
 - `c2.md` — Arquitectura y modelo de datos
 - `c3.md` — Implementación de la API
 - `c4.md` — Autenticación y Autorización
- **mysql/**
 - `init.sql` — Esquema + 30 registros por tabla
- **postman/**
 - `taskmanager-api.postman_collection.json`
- **src/**
 - **config/** → `db.js` (pool de conexión a MySQL)
 - **controllers/** → `auth.controller.js`, `users.controller.js`, `projects.controller.js`, `tasks.controller.js`
 - **middlewares/** → `auth.middleware.js` (verificación del JWT)
 - **routes/** → `auth.routes.js`, `users.routes.js`, `projects.routes.js`, `tasks.routes.js`
 - `index.js` — Entry point de la aplicación
- `.env.example` — Plantilla de variables de entorno
- `.gitignore`
- `Dockerfile` — Imagen Node de la API
- `docker-compose.yml` — Orquestación MySQL + API
- `package.json`
- `README.md`

1.5. Equipo

- **Marcos Martínez Fernández** — A048665
- **Francisco Capilla Sánchez** — A048669

Repositorio: <https://github.com/inf25dw2g05/taskmanager>

Capítulo 2 — Arquitectura y modelo de datos

2.1. Arquitectura general

La aplicación sigue una **arquitectura de capas** desplegada en **dos contenedores Docker** orquestados con Docker Compose. La comunicación entre componentes es la siguiente:

Componente	Tipo	Comunicación
Cliente (Postman, navegador, etc.)	Externo	Envía peticiones HTTP con JSON y cabecera <code>Authorization: Bearer <JWT></code> al contenedor de la API
taskmanager-app	Contenedor Node.js 20 + Express 5	Recibe peticiones HTTP en el puerto 3000, las procesa y consulta MySQL
taskmanager-mysql	Contenedor MySQL 8.0	Recibe consultas SQL del contenedor de la API por el puerto 3306 (interno)
Red taskmanager-net	Red interna de Docker	Permite que los contenedores se comuniquen entre sí usando nombres de servicio (<code>mysql:3306</code>) en lugar de IPs
Volumen mysql_data	Volumen Docker	Garantiza la persistencia de los datos aunque el contenedor de MySQL se reinicie

El cliente externo solo accede al puerto 3000 de la API. MySQL queda aislado dentro de la red de Docker (aunque se expone el puerto 3307 en el host para facilitar inspección durante el desarrollo).

2.2. Estructura del código (capas)

Dentro del contenedor de la aplicación, el código está dividido en cuatro capas con responsabilidades claras:

Capa	Carpeta	Responsabilidad
Routes	<code>src/routes/</code>	Definen los endpoints HTTP y los enlazan con los controladores.
Middlewares	<code>src/middlewares/</code>	Funciones intermedias: verificación de JWT y logging.
Controllers	<code>src/controllers/</code>	Lógica de negocio: validación, consultas SQL y respuestas.
Config	<code>src/config/</code>	Conexión a MySQL mediante un pool reutilizable.

Este patrón es una variante simplificada de **MVC** (Model-View-Controller) adaptada a una API REST sin vistas. La separación facilita el mantenimiento: añadir un nuevo recurso supone añadir un fichero en `routes/`, otro en `controllers/`, y registrar la ruta en `index.js`.

2.3. Flujo de una petición protegida

Tomando como ejemplo `GET /projects`, el flujo completo es:

1. **Ciente** envía la petición con la cabecera `Authorization: Bearer <token>`.
2. **Express** la enruta hacia `projects.routes.js`.
3. Antes de llegar al controlador, el **middleware** `authMiddleware` intercepta la petición:
 - Extrae el token de la cabecera.
 - Lo verifica con `jwt.verify()` usando la clave secreta del servidor.
 - Si es válido, adjunta el payload del token a `req.user` (id, username, email).
 - Si no es válido o falta, responde con `401` o `403` y corta el flujo.
4. El controlador `projects.controller.js` ejecuta la query SQL **filtrando por** `req.user.id`, de modo que la base de datos solo devuelve los proyectos del usuario autenticado.
5. La respuesta se serializa en **JSON** y se devuelve al cliente.

Adicionalmente, el middleware imprime en la consola del servidor los detalles del usuario autenticado en cada petición protegida — requisito explícito del enunciado:

2.4. Modelo de datos

El modelo está compuesto por **tres entidades** con **dos relaciones 1:n** entre ellas.

2.4.1. Entidades

Tabla users — representa a los usuarios del sistema:

Columna	Tipo	Restricciones	Descripción
<code>id</code>	INT	PK, AUTO_INCREMENT	Identificador único del usuario
<code>username</code>	VARCHAR(100)	NOT NULL	Nombre visible del usuario
<code>email</code>	VARCHAR(150)	NOT NULL, UNIQUE	Email (usado como login)
<code>password</code>	VARCHAR(255)	NOT NULL	Contraseña hasheada con bcrypt
<code>created_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de creación

Tabla projects — proyectos que pertenecen a un usuario:

Columna	Tipo	Restricciones	Descripción
<code>id</code>	INT	PK, AUTO_INCREMENT	Identificador único del proyecto
<code>name</code>	VARCHAR(150)	NOT NULL	Nombre del proyecto
<code>description</code>	TEXT	—	Descripción libre
<code>user_id</code>	INT	NOT NULL, FK → <code>users.id</code>	Propietario del proyecto
<code>created_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de creación

Tabla tasks — tareas que pertenecen a un proyecto:

Columna	Tipo	Restricciones	Descripción
<code>id</code>	INT	PK, AUTO_INCREMENT	Identificador único de la tarea

Columna	Tipo	Restricciones	Descripción
title	VARCHAR(150)	NOT NULL	Título corto de la tarea
description	TEXT	—	Descripción libre
completed	BOOLEAN	DEFAULT FALSE	Indica si la tarea está completada
project_id	INT	NOT NULL, FK → projects.id	Proyecto al que pertenece
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de creación

2.4.2. Relaciones

Relación	Cardinalidad	Implementación	Comportamiento al eliminar
users → projects	1:n	projects.user_id referencia users.id	ON DELETE CASCADE: al borrar un usuario, se borran sus proyectos
projects → tasks	1:n	tasks.project_id referencia projects.id	ON DELETE CASCADE: al borrar un proyecto, se borran sus tareas

El comportamiento en cascada garantiza la **integridad referencial** y evita registros huérfanos en la base de datos.

2.4.3. Esquema SQL completo

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE projects (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(150) NOT NULL,  
  description TEXT,  
  user_id INT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);  
  
CREATE TABLE tasks (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(150) NOT NULL,  
  description TEXT,  
  completed BOOLEAN DEFAULT FALSE,  
  project_id INT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE CASCADE
);
```

2.4.4. Decisiones de diseño relevantes

- **email UNIQUE** en la tabla **users** — garantiza que no haya dos cuentas con el mismo correo y permite usarlo como identificador en el login.
- **password VARCHAR(255)** — el hash bcrypt ocupa 60 caracteres, pero se reserva más espacio por previsión (otros algoritmos, prefijos largos).
- **completed BOOLEAN DEFAULT FALSE** — toda tarea nueva nace como pendiente, simplificando la creación desde el cliente.
- **created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP** en todas las tablas — permite ordenar por fecha de creación sin que el cliente tenga que enviar la fecha.
- **No se ha implementado updated_at** — el alcance del M1 no lo requería; se podría añadir en futuras iteraciones.

2.5. Datos de prueba

El fichero **mysql/init.sql** carga automáticamente los siguientes datos al inicializar el contenedor de MySQL por primera vez:

- **30 usuarios** (3 destacados como **demo1**, **demo2**, **demo3** para facilitar la presentación).
- **30 proyectos** distribuidos entre los primeros 27 usuarios.
- **30 tareas** distribuidas entre los proyectos, mezclando completadas y pendientes para ilustrar diferentes estados.

Todas las contraseñas en el seed son **demo1234** (hasheadas con bcrypt en formato **\$2b\$10\$...**). Esto cumple holgadamente con el requisito del enunciado de "al menos 30 registros por tabla".

2.6. Despliegue con Docker Compose

El fichero **docker-compose.yml** define los dos servicios:

- **mysql**: usa la imagen oficial **mysql:8.0**. Monta el script **init.sql** como volumen en **/docker-entrypoint-initdb.d/**, donde MySQL lo ejecuta automáticamente la primera vez que arranca. Los datos se persisten en un volumen llamado **mysql_data**. Expone el puerto 3306 al host como 3307 para evitar conflictos con instalaciones locales de MySQL.
- **app**: se construye a partir del **Dockerfile** del repositorio. Depende de **mysql** con la condición **service_healthy**, gracias a un **healthcheck** que comprueba que MySQL responde antes de arrancar Node. Expone el puerto 3000.

Ambos servicios comparten una red **taskmanager-net** y reciben sus credenciales mediante un fichero **.env** (no versionado, plantilla en **.env.example**).

Capítulo 3 — Implementación de la API

3.1. Visión general de los endpoints

La API expone **15 endpoints** organizados en 4 grupos lógicos. La siguiente tabla resume todos los endpoints, los verbos HTTP utilizados y si requieren autenticación:

Grupo	Método	Endpoint	Descripción	Auth
Auth	POST	<code>/auth/register</code>	Registrar un nuevo usuario	No
Auth	POST	<code>/auth/login</code>	Iniciar sesión y obtener token JWT	No
Users	GET	<code>/users</code>	Listar todos los usuarios	Sí
Users	GET	<code>/users/me</code>	Obtener el perfil del usuario autenticado	Sí
Users	GET	<code>/users/:id</code>	Obtener un usuario por id	Sí
Users	PUT	<code>/users/me</code>	Actualizar el perfil del usuario autenticado	Sí
Users	DELETE	<code>/users/me</code>	Eliminar la cuenta del usuario autenticado	Sí
Projects	GET	<code>/projects</code>	Listar proyectos del usuario autenticado	Sí
Projects	GET	<code>/projects/:id</code>	Obtener un proyecto por id	Sí
Projects	POST	<code>/projects</code>	Crear un proyecto	Sí
Projects	PUT	<code>/projects/:id</code>	Actualizar un proyecto	Sí
Projects	DELETE	<code>/projects/:id</code>	Eliminar un proyecto	Sí
Tasks	GET	<code>/tasks</code>	Listar tareas del usuario autenticado	Sí
Tasks	GET	<code>/tasks/:id</code>	Obtener una tarea por id	Sí
Tasks	POST	<code>/tasks</code>	Crear una tarea	Sí
Tasks	PUT	<code>/tasks/:id</code>	Actualizar una tarea	Sí
Tasks	DELETE	<code>/tasks/:id</code>	Eliminar una tarea	Sí

Se cumple el requisito del enunciado de utilizar los **cuatro verbos HTTP (GET, POST, PUT, DELETE)** para implementar las operaciones CRUD sobre tres recursos diferentes.

3.2. Arranque de la aplicación

El fichero `src/index.js` es el punto de entrada. Su responsabilidad es:

1. Cargar variables de entorno desde `.env`.
2. Inicializar Express y los middlewares globales (`cors`, `express.json`).
3. Inicializar la conexión a MySQL (importando `src/config/db.js`).
4. Montar Swagger UI en `/docs` para servir la documentación OpenAPI.

5. Registrar las rutas de cada grupo, aplicando el middleware de autenticación donde corresponde.
6. Arrancar el servidor en el puerto 3000.

Las rutas se registran de la siguiente forma:

```
app.use("/auth", authRoutes);
app.use("/users", authMiddleware, usersRoutes);
app.use("/projects", authMiddleware, projectsRoutes);
app.use("/tasks", authMiddleware, tasksRoutes);
```

`/auth` es la única ruta pública. El resto pasa por `authMiddleware` antes de llegar al controlador.

3.3. Conexión a MySQL: pool de conexiones

El fichero `src/config/db.js` configura un **pool** de conexiones a MySQL en lugar de una conexión única. Esta decisión es importante por tres razones:

1. **Tolerancia a fallos:** si una conexión se cae (por ejemplo, durante el arranque inicial mientras MySQL aún se está inicializando), el pool descarta esa conexión y crea otra automáticamente.
2. **Concurrencia:** múltiples peticiones HTTP pueden ejecutarse en paralelo sin esperar a que la conexión anterior termine.
3. **Estabilidad:** con una conexión única, una desconexión deja la aplicación en estado inutilizable hasta reiniciar. Con un pool, la API se autoregenera.

```
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: process.env.DB_PORT,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});
```

La API de `pool.query()` es idéntica a la de `connection.query()`, por lo que el cambio fue transparente para los controladores.

3.4. Patrón Routes + Controllers

Cada grupo de recursos sigue el mismo patrón: un fichero de rutas que enlaza endpoints con funciones del controlador, y un fichero de controlador con la lógica de negocio.

Ejemplo: rutas de Projects

```
// src/routes/projects.routes.js
const express = require("express");
```

```
const router = express.Router();
const projectsController = require("../controllers/projects.controller");

router.get("/", projectsController.getAllProjects);
router.get("/:id", projectsController.getProjectById);
router.post("/", projectsController.createProject);
router.put("/:id", projectsController.updateProject);
router.delete("/:id", projectsController.deleteProject);

module.exports = router;
```

El router solo se encarga de **mapear**: ningún tipo de lógica de negocio vive aquí. Esto facilita ver de un vistazo qué endpoints expone cada recurso.

Ejemplo: controlador de Projects

```
// src/controllers/projects.controller.js – extracto
function getAllProjects(req, res) {
  const userId = req.user.id;

  db.query(
    "SELECT * FROM projects WHERE user_id = ?",
    [userId],
    function(err, results) {
      if (err) {
        res.status(500).json(err);
        return;
      }
      res.json(results);
    }
  );
}
```

Observaciones clave:

- **req.user.id** proviene del JWT decodificado por el middleware. Está garantizado que existe porque la ruta está protegida.
- **Consultas parametrizadas** (?) y array de valores) para prevenir inyección SQL. Es lo que recomienda **mysql2** y es práctica obligatoria de seguridad.
- **Filtrado por propietario** directamente en la cláusula **WHERE**: la base de datos solo devuelve los proyectos del usuario que ha hecho la petición.

3.5. Operaciones CRUD detalladas

Cada recurso implementa las 5 operaciones típicas. A continuación se describe brevemente cómo se ha implementado cada una para el recurso **Projects** (las otras siguen un patrón análogo).

3.5.1. Listar (GET /projects)

```
SELECT * FROM projects WHERE user_id = ?
```

Devuelve únicamente los proyectos del usuario autenticado. Status **200 OK** con un array JSON.

3.5.2. Obtener uno (GET /projects/:id)

```
SELECT * FROM projects WHERE id = ? AND user_id = ?
```

El doble filtro (**id** y **user_id**) garantiza que un usuario no pueda obtener proyectos ajenos aunque conozca su id. Si la consulta no devuelve resultados, se responde con **404 Not Found** y el mensaje "Proyecto no encontrado o sin permisos".

Decisión de diseño: se devuelve **404** en lugar de **403** deliberadamente. Esto evita que un atacante pueda inferir la existencia de proyectos ajenos mediante la diferencia entre "no existe" (**404**) y "no tienes permiso" (**403**).

3.5.3. Crear (POST /projects)

```
INSERT INTO projects (name, description, user_id) VALUES (?, ?, ?)
```

El **user_id** **no se acepta del cliente**: se toma directamente de **req.user.id**. Esto impide que un usuario malicioso intente crear proyectos en nombre de otro. Status **201 Created** y se devuelve el **projectId** generado.

3.5.4. Actualizar (PUT /projects/:id)

```
UPDATE projects SET name = ?, description = ? WHERE id = ? AND user_id = ?
```

De nuevo, el doble filtro evita la modificación de proyectos ajenos. Si **affectedRows === 0**, se responde con **404**.

3.5.5. Eliminar (DELETE /projects/:id)

```
DELETE FROM projects WHERE id = ? AND user_id = ?
```

Por el **ON DELETE CASCADE** definido en el esquema, eliminar un proyecto elimina también todas sus tareas automáticamente.

3.6. Caso especial: tareas anidadas bajo proyectos

El recurso **Tasks** tiene una particularidad: aunque sus endpoints son `/tasks`, las tareas siempre pertenecen a un proyecto. Para garantizar el aislamiento por usuario, las queries hacen **JOIN** con la tabla **projects**:

```
SELECT tasks.*
FROM tasks
INNER JOIN projects ON tasks.project_id = projects.id
WHERE projects.user_id = ?
```

Esto significa que un usuario solo verá tareas cuyo proyecto le pertenezca. La verificación se hace **a nivel de base de datos**, no en código Node, lo que es más eficiente y más difícil de eludir.

En la operación de **creación** de tareas se hace una comprobación previa:

```
// Verificar que el project_id indicado pertenece al usuario
db.query(
  "SELECT * FROM projects WHERE id = ? AND user_id = ?",
  [project_id, userId],
  function(err, projects) {
    if (projects.length === 0) {
      res.status(403).json({
        message: "No puedes crear tareas en un proyecto que no es tuyo"
      });
      return;
    }
    // ... INSERT INTO tasks ...
  }
);
```

Si el usuario intenta crear una tarea en un proyecto ajeno, se responde con **403 Forbidden**.

3.7. Endpoints `/me` frente a `/:id`

En el recurso **Users** se ha optado por exponer endpoints `/users/me` para las operaciones que afectan al usuario autenticado:

Endpoint genérico	Endpoint <code>/me</code>
PUT <code>/users/:id</code> (peligroso)	PUT <code>/users/me</code> (seguro)
DELETE <code>/users/:id</code> (peligroso)	DELETE <code>/users/me</code> (seguro)

La razón es que **PUT** `/users/:id` permitiría a un usuario intentar modificar a otro pasando un id ajeno. Con **PUT** `/users/me`, el `id` se extrae del token JWT y no del path, por lo que es imposible modificar a un usuario que no sea uno mismo.

GET `/users/:id` se mantiene porque devuelve información pública (sin password) y es razonable que un usuario pueda consultar perfiles ajenos.

Detalle de implementación: en `users.routes.js`, las rutas `/me` se declaran **antes** que `/:id`. Si se hiciera al revés, Express interpretaría la cadena `"me"` como un valor de `:id` numérico, llamaría al controlador `getUserById` y la búsqueda fallaría.

3.8. Formato de respuestas

Todas las respuestas son **JSON** (cabecera `Content-Type: application/json; charset=utf-8`). El formato sigue un convenio simple:

- **Éxito con datos:** objeto JSON o array de objetos directamente.
- **Éxito sin datos:** `{ "message": "..."}.`
- **Error:** `{ "message": "...", "error": ... }` con el status code HTTP correspondiente.

Códigos de status utilizados

Código	Significado	Casos de uso
200 OK	Operación exitosa	GET, PUT, DELETE
201 Created	Recurso creado	POST
401 Unauthorized	Falta token o es inválido	Cualquier endpoint protegido sin auth
403 Forbidden	Token válido pero sin permiso	Crear tarea en proyecto ajeno
404 Not Found	Recurso inexistente o sin acceso	GET/PUT/DELETE de id inexistente
500 Internal Server Error	Error inesperado	Fallos de BD o de servidor

3.9. Documentación con OpenAPI 3.0

La documentación de la API se encuentra en `api/openapi.yaml`, conforme a la especificación **OpenAPI 3.0**. Se sirve a través de **Swagger UI** en el endpoint `/docs`, gracias al paquete `swagger-ui-express` montado en `index.js`:

```
const swaggerDocument = YAML.load("./api/openapi.yaml");
app.use("/docs", swaggerUi.serve, swaggerUi.setup(swaggerDocument));
```

La especificación incluye:

- **Tags** para agrupar endpoints por recurso (Auth, Users, Projects, Tasks).
- **Schemas reutilizables** definidos en `components/schemas` (User, Project, Task, RegisterRequest, etc.).
- **Parámetros reutilizables** (como `IdParam`) en `components/parameters`.
- **Respuestas reutilizables** (`Unauthorized`, `NotFound`, `ServerError`) en `components/responses`.
- **Esquema de seguridad `bearerAuth`** aplicado a nivel global, con sobreescritura en los endpoints públicos (`/auth/register`, `/auth/login`).
- **Ejemplos** de request bodies y de respuestas en cada endpoint.

Swagger UI permite además **probar los endpoints desde el navegador**: tras hacer login y obtener un token, se pulsa el botón "Authorize" para introducirlo y a partir de ahí cualquier endpoint protegido puede ejecutarse desde la propia documentación.

3.10. Colección Postman

Se ha preparado una colección Postman ([postman/taskmanager-api.postman_collection.json](#)) que cubre todos los endpoints organizados en las 4 carpetas. La colección incluye:

- **Variables de entorno** ([baseUrl](#), [token](#)) configuradas a nivel de colección.
 - **Autenticación Bearer heredada** desde la raíz de la colección: ninguna petición protegida necesita configurar la cabecera manualmente.
 - **Script automático en el endpoint Login**: tras hacer login con éxito, el token JWT recibido se almacena automáticamente en la variable `{{token}}`, de modo que todas las llamadas posteriores lo utilizan sin intervención del usuario.
 - **Tests automáticos** en el endpoint Login que verifican el código 200 y la presencia del token en la respuesta.
 - **Login alternativo con un segundo usuario** ([demo2](#)) para demostrar de forma visual el aislamiento por propietario: con un par de clics se puede comprobar que cada usuario ve únicamente sus proyectos y tareas.
-

Capítulo 4 — Autenticación y Autorización

Este capítulo detalla la implementación de la capa de autenticación y autorización de TaskManager API y la compara con los distintos flujos del estándar **OAuth 2.0**, tal y como solicita el enunciado.

4.1. Conceptos previos: autenticación vs autorización

Aunque a menudo se usan como sinónimos, son dos conceptos distintos:

- **Autenticación** (*authentication*): verificar **quién es** el usuario. Responde a la pregunta "*¿eres quien dices ser?*". El mecanismo típico es la comprobación de credenciales (email + contraseña).
- **Autorización** (*authorization*): verificar **qué puede hacer** un usuario ya autenticado. Responde a la pregunta "*¿tienes permiso para hacer esta operación sobre este recurso?*". En TaskManager se traduce en: "*¿este proyecto te pertenece?*".

La API implementa ambas capas: autenticación en `/auth/login` (devuelve un token) y autorización en cada endpoint protegido (verifica el token y filtra los recursos por propietario).

4.2. Mecanismo implementado: JWT

TaskManager utiliza **JSON Web Tokens (JWT)** como mecanismo de autenticación stateless. El flujo es:

1. El cliente envía credenciales (email + contraseña) a `POST /auth/login`.
2. El servidor verifica las credenciales contra la base de datos:
 - Busca al usuario por email.
 - Compara la contraseña enviada con el hash almacenado usando `bcrypt.compareSync()`.
3. Si las credenciales son válidas, el servidor genera un **JWT firmado** con su clave secreta y lo devuelve.
4. El cliente almacena el token y lo envía en cada petición posterior en la cabecera `Authorization: Bearer <token>`.
5. El servidor verifica el token en cada petición protegida (middleware) y extrae el `userId` del payload.

4.2.1. Hash de contraseñas con bcrypt

Las contraseñas **nunca** se almacenan en texto plano. Se utiliza el algoritmo `bcrypt` con factor de coste 10:

```
// Registro
const hashedPassword = bcrypt.hashSync(password, 10);
db.query("INSERT INTO users (... , password) VALUES (... , ?)", [... ,
hashedPassword]);

// Login
const validPassword = bcrypt.compareSync(password, user.password);
```

`bcrypt` añade automáticamente un *salt* aleatorio al hash, por lo que dos contraseñas idénticas producen hashes distintos. Esto protege frente a ataques de tablas rainbow y dificulta los ataques por fuerza bruta.

4.2.2. Estructura del token JWT

Un JWT está compuesto por tres partes separadas por puntos, codificadas en Base64URL:

El **payload** que se firma en TaskManager contiene:

```
{
  id: user.id,
  username: user.username,
  email: user.email,
  iat: <timestamp de emisión>,
  exp: <timestamp de expiración: 1 hora después>
}
```

La generación:

```
const token = jwt.sign(
  { id: user.id, username: user.username, email: user.email },
  process.env.JWT_SECRET,
  { expiresIn: "1h" }
);
```

La **firma** se calcula como `HMAC-SHA256(base64(header) + "." + base64(payload), JWT_SECRET)`. Sin conocer `JWT_SECRET`, es computacionalmente inviable falsificar un token, ya que cualquier modificación del payload invalida la firma.

4.2.3. Verificación del token (middleware)

El middleware `auth.middleware.js` se encarga de validar el token en cada petición protegida:

```
function authMiddleware(req, res, next) {
  const authHeader = req.headers["authorization"];

  if (!authHeader) {
    res.status(401).json({ message: "Token no proporcionado" });
    return;
  }

  const token = authHeader.split(" ")[1];

  jwt.verify(token, process.env.JWT_SECRET, function(err, user) {
    if (err) {
      res.status(403).json({ message: "Token no válido" });
      return;
    }
    req.user = user;
    console.log("Usuario autenticado:", req.user);
    next();
  });
}
```

```
});  
}
```

`jwt.verify()` realiza tres comprobaciones:

1. Que la firma sea válida (token no manipulado).
2. Que el token no haya expirado (`exp` no superado).
3. Que el algoritmo de firma coincida con el esperado.

Si todas pasan, el payload se adjunta a `req.user`, queda disponible para los controladores y la petición continúa.

4.3. Control de acceso por propietario

La autenticación garantiza saber **quién** hace la petición. La autorización determina **qué** puede hacer. En TaskManager, el control de acceso se aplica a nivel de **propietario del recurso**:

- En `projects`: el filtro `WHERE user_id = ?` con el id del token impide acceder a proyectos ajenos.
- En `tasks`: el `JOIN` con `projects` filtrando por `user_id` impide acceder a tareas de proyectos ajenos.
- En `users/me`: el id se extrae del token, no del path, por lo que es imposible modificar a otro usuario.

Este es el cumplimiento del requisito valorado del enunciado: "O utilizador autenticado apenas consegue aceder aos seus próprios recursos".

4.4. Comparación con los flujos de OAuth 2.0

OAuth 2.0 es un **estándar de autorización** definido en la [RFC 6749](#). No es un protocolo de autenticación, sino un marco para delegar acceso a recursos protegidos. Define varios *flows* (también llamados *grant types*), cada uno pensado para un caso de uso específico.

A continuación se comparan los principales flujos con la implementación actual de TaskManager.

4.4.1. Roles de OAuth 2.0

OAuth 2.0 define cuatro roles:

Rol	Definición	Equivalente en TaskManager
Resource Owner	El usuario final, propietario de los datos	El usuario <code>demo1</code> , <code>demo2</code> , etc.
Resource Server	Servidor que aloja los recursos protegidos	La API TaskManager
Authorization Server	Servidor que emite tokens	El propio TaskManager (no está separado)
Client	Aplicación que accede a los recursos	Postman, Swagger UI, frontend

En OAuth 2.0 estricto, el Authorization Server y el Resource Server son entidades **separadas**. En TaskManager, ambos roles los desempeña la misma aplicación, lo que simplifica la arquitectura pero limita la federación con

terceros.

4.4.2. Authorization Code Flow

Es el flujo más utilizado y seguro de OAuth 2.0. Pensado para aplicaciones web con backend.

Pasos del flujo:

1. El usuario hace clic en "*Iniciar sesión con Google*".
2. El cliente redirige al usuario al Authorization Server (Google).
3. El usuario se autentica en Google e **autoriza** al cliente.
4. Google redirige al cliente con un *authorization code* temporal.
5. El cliente intercambia ese code (junto con su *client_id* y *client_secret*) por un **access token**.
6. El cliente usa el access token para acceder a la API.

Diferencias con TaskManager:

Aspecto	Authorization Code Flow	TaskManager (JWT)
Quién autentica	Tercero (Google, GitHub, etc.)	El propio servidor TaskManager
Almacena contraseñas	No (delega en el tercero)	Sí (con bcrypt)
Número de pasos	6 (incluye redirecciones)	2 (login → token)
Adecuado para	Delegar identidad a un tercero	Aplicación con cuentas propias
Complejidad	Alta	Baja

Este flujo sería el adecuado si TaskManager quisiera ofrecer "*login con Google*" en lugar de gestionar contraseñas propias. Es el flow recomendado para producción cuando interesa delegar la gestión de identidad.

4.4.3. Resource Owner Password Credentials Flow

Este flujo permite que el cliente intercambie directamente el email y la contraseña del usuario por un token. **Es el más parecido al implementado en TaskManager.**

Pasos del flujo:

1. El cliente solicita las credenciales al usuario.
2. El cliente envía las credenciales al Authorization Server.
3. El Authorization Server las verifica y emite un access token.

Comparación con TaskManager:

Aspecto	Password Credentials Flow	TaskManager (JWT)
Envío de credenciales	Sí, al Authorization Server	Sí, a <i>/auth/login</i>
Devuelve token	Sí (access token, opcionalmente refresh token)	Sí (JWT)
Token autocontenido	No necesariamente	Sí (JWT contiene el payload)

Aspecto	Password Credentials Flow	TaskManager (JWT)
Refresh token	Sí	No implementado

La implementación de TaskManager es **conceptualmente equivalente** a este flow, con las siguientes diferencias técnicas:

- TaskManager **no implementa refresh tokens**. Cuando el JWT expira (1 hora), el usuario debe hacer login de nuevo. En un escenario real se añadiría un endpoint `/auth/refresh` que devolviera un nuevo access token a partir de un refresh token de larga duración.
- El token de TaskManager es un **JWT autocontenido** (stateless), mientras que en OAuth 2.0 clásico el token puede ser opaco y requerir consulta al Authorization Server (introspection) para validarlo.

Nota: el propio estándar OAuth 2.0 desaconseja este flujo (RFC 6749 §1.3.3) porque obliga al cliente a manejar credenciales en texto plano, lo que rompe el principio de delegación. Actualmente se considera *legacy*. Sin embargo, sigue siendo el modelo más común en APIs con cuentas propias y sin federación, como es el caso de TaskManager en este M1.

4.4.4. Client Credentials Flow

Este flujo está pensado para comunicación **máquina a máquina**, sin usuario final involucrado.

Pasos del flujo:

1. El cliente (otro servicio backend) envía sus propias credenciales (`client_id` + `client_secret`).
2. El Authorization Server emite un access token asociado al cliente, no a un usuario.

Comparación con TaskManager:

No es aplicable al caso de uso. TaskManager está diseñado para usuarios humanos finales, no para integraciones entre servicios. Si en el futuro se quisiera permitir que otra aplicación accediera a TaskManager *en nombre propio* (por ejemplo, para tareas de mantenimiento o sincronización), este sería el flow indicado.

4.4.5. Implicit Flow (obsoleto)

Pensado originalmente para aplicaciones SPA (single-page applications) sin backend, donde el token se devolvía directamente en la URL.

Estado actual: la [OAuth 2.0 Security Best Current Practice \(RFC 9700\)](#) lo declara **obsoleto** por riesgos de fuga del token (queda registrado en historial del navegador, logs, etc.). Para SPAs hoy se recomienda **Authorization Code + PKCE**.

No es aplicable a TaskManager y no se considera una alternativa válida.

4.4.6. Authorization Code + PKCE

Variante del Authorization Code Flow diseñada para clientes públicos (móviles, SPAs) que no pueden almacenar de forma segura un `client_secret`.

Diferencia clave con Authorization Code: en lugar de un `client_secret`, el cliente genera un `code verifier` aleatorio y envía su hash (`code challenge`) en la petición inicial. Al canjear el code por el token, debe presentar el `code verifier` original, demostrando que es el mismo cliente que inició el flow.

Comparación con TaskManager: si TaskManager evolucionara a tener un frontend SPA (React, Vue) y se integrara con un proveedor externo de identidad (Google, Auth0), este sería el flow recomendado actualmente.

4.5. Tabla resumen comparativa

Flow OAuth 2.0	Caso de uso típico	Aplicable a TaskManager	Por qué
Authorization Code	Web con backend, login federado	Como mejora futura	Si se quisiera "login con Google"
Authorization Code + PKCE	SPAs, apps móviles	Como mejora futura	Si se hiciera frontend SPA
Resource Owner Password Credentials	API con cuentas propias	El más cercano al actual	Es lo que esencialmente hace /auth/login
Client Credentials	Comunicación máquina-máquina	No aplicable	No hay clientes no-humanos
Implicit	(Obsoleto)	No aplicable	Inseguro, deprecado por la RFC 9700

4.6. Justificación de la elección actual y mejoras futuras

Para el alcance del M1, la implementación con JWT + bcrypt es:

- **Suficiente** para cubrir todos los requisitos obligatorios del enunciado (autenticación, autorización, control por propietario, log del usuario).
- **Simple:** no requiere infraestructura adicional (Authorization Server externo, almacenamiento de tokens en BD, etc.).
- **Conceptualmente alineada** con el flow *Resource Owner Password Credentials* de OAuth 2.0.

Como **mejoras futuras**, fuera del alcance del M1, se podrían implementar:

1. **Refresh tokens:** para evitar que el usuario tenga que hacer login cada hora. Se almacenarían en la base de datos con su `user_id` y fecha de expiración (típicamente días o semanas).
2. **OAuth 2.0 con proveedor externo:** integrar Passport.js + estrategia de Google/GitHub para ofrecer login federado siguiendo el Authorization Code Flow.
3. **Roles y permisos:** ampliar el payload del token con un campo `role` (admin, user, etc.) y un middleware adicional de autorización por rol.
4. **Revocación de tokens:** en JWT stateless no es posible invalidar tokens antes de su expiración. Para soportarlo, se introduciría una lista negra (`blacklist`) en Redis o se migraría a tokens opacos con introspección.
5. **Rotación de claves:** rotar `JWT_SECRET` periódicamente con sistemas de key versioning.

4.7. Conclusión

La capa de autenticación y autorización implementada en TaskManager cubre los requisitos del enunciado mediante un esquema **JWT + bcrypt** sencillo, robusto y bien establecido en la industria. Conceptualmente, se

sitúa próxima al flow *Resource Owner Password Credentials* de OAuth 2.0.

El uso de JWT stateless permite escalar horizontalmente la aplicación sin necesidad de almacenar sesiones, y la verificación del token en cada petición protegida (mediante middleware) garantiza tanto la autenticación como el log de auditoría que el enunciado solicita.

La adopción de un Authorization Server externo (Google, Auth0, etc.) mediante Authorization Code + PKCE sería el siguiente paso natural si el proyecto evolucionara hacia un entorno de producción con federación de identidad.
